Project 3: Hierarchical Gossip Aggregation for Decentralized Federated Learning with Domain Clustering

Timeline: 9-11 months
Target Venues: NeurIPS, ICML, ICLR
Status: Standalone Research Project

Executive Summary

This project presents a novel **hierarchical gossip aggregation algorithm** that enables efficient peer-to-peer communication in decentralized federated learning while explicitly accounting for domain heterogeneity. Unlike traditional gossip-based methods that treat all clients equally, our approach clusters clients by domain similarity and performs hierarchical aggregation that preserves domain-specific knowledge while enabling cross-domain transfer.

The key innovation is a **domain-aware gossip protocol** that:

1. Clusters clients into domain groups based on LoRA matrix similarity
2. Performs gossip aggregation within domain clusters
3. Learns cross-domain transfer weights for inter-cluster communication
4. Ensures convergence guarantees in decentralized settings

Key Results:

- 20-25% reduction in communication rounds compared to standard gossip
- 15-20% accuracy improvement over domain-unaware gossip
- Convergence guarantees with theoretical analysis
- Scalable to 100+ clients in decentralized networks

Problem Statement

The Challenge: Decentralized Federated Learning with Heterogeneous Domains

Decentralized federated learning eliminates the central server, enabling direct peer-to-peer communication. However, this creates new challenges:

1. **No Global Coordinator**: No central entity to orchestrate aggregation
2. **Communication Efficiency**: Every communication is expensive in peer-to-peer networks
3. **Convergence Guarantees**: Must prove convergence without central coordination
4. **Domain Heterogeneity**: Different clients have different data distributions

Existing Gossip-Based Approaches Fail:

- Standard gossip averages all clients equally, ignoring domain structure
- Results in mediocre performance on all domains
- Slow convergence due to inefficient communication patterns
- No theoretical guarantees with domain heterogeneity

Real-World Scenario: Hospital Network

Imagine 12 hospitals in a peer-to-peer network wanting to collaboratively improve diagnostic AI:

```
Cardiology Hospitals:
  H1 (Cardiac) ↔ H2 (Cardiac) ↔ H3 (Cardiac)

Oncology Hospitals:
  H4 (Tumor) ↔ H5 (Tumor) ↔ H6 (Tumor)

Neurology Hospitals:
  H7 (Brain) ↔ H8 (Brain) ↔ H9 (Brain)

Inter-domain connections:
  H3 ↔ H4  (Cardiology ↔ Oncology)
  H6 ↔ H7  (Oncology ↔ Neurology)
  H9 ↔ H1  (Neurology ↔ Cardiology)
```

Standard Gossip: Each hospital communicates with neighbors randomly, averaging all updates equally. Result: Poor performance on all domains.

Our Approach:

- Within-domain gossip: H1-H2-H3 aggregate among themselves
- Cross-domain gossip: H3 communicates with H4, learning what's transferable
- Result: Each domain excels while benefiting from other domains

---

Technical Approach

Component 1: Domain-Aware Network Topology

1.1 Client Clustering by Domain Similarity

```
def cluster_clients_by_domain_similarity(
    client_lora_matrices,
    n_clusters=None,
    similarity_metric='cosine'
):
    """
    Cluster clients into domain groups based on LoRA matrix similarity

    Intuition: Clients in the same domain learn similar LoRA patterns
    """

    # Extract features from LoRA matrices
    features = []
    client_ids = []

    for client_id, (A, B) in client_lora_matrices.items():
        # Use SVD to get low-dimensional representation
        _, s_a, _ = np.linalg.svd(A, full_matrices=False)
        _, s_b, _ = np.linalg.svd(B, full_matrices=False)

        # Concatenate singular values as feature vector
        feature = np.concatenate([s_a, s_b])
        features.append(feature)
        client_ids.append(client_id)

    features = np.array(features)

    # Normalize features
    features = (features - features.mean(axis=0)) / (features.std(axis=0) +
1e-10)

    # Determine optimal number of clusters if not specified
    if n_clusters is None:
        n_clusters = estimate_optimal_clusters(features)

    # Hierarchical clustering
    clustering = AgglomerativeClustering(
```

```
        n_clusters=n_clusters,
        linkage='ward'
    )
    cluster_assignments = clustering.fit_predict(features)

    # Group clients by cluster
    domain_clusters = {}
    for client_id, cluster_id in zip(client_ids, cluster_assignments):
        if cluster_id not in domain_clusters:
            domain_clusters[cluster_id] = []
        domain_clusters[cluster_id].append(client_id)

    return domain_clusters, cluster_assignments
```

1.2 Build Hierarchical Network Topology

```
def build_hierarchical_network_topology(
    domain_clusters,
    network_topology='ring'
):
    """
    Build hierarchical network topology:
    1. Within-domain: Ring or star topology
    2. Between-domain: Connect cluster representatives
    """

    network = {
        'within_domain': {},
        'between_domain': {}
    }

    # Step 1: Build within-domain topology
    for cluster_id, clients in domain_clusters.items():
        if network_topology == 'ring':
            # Ring topology: C1 ↔ C2 ↔ C3 ↔ ... ↔ C1
            for i, client in enumerate(clients):
                next_client = clients[(i + 1) % len(clients)]
                prev_client = clients[(i - 1) % len(clients)]

                network['within_domain'][client] = [prev_client,
next_client]

        elif network_topology == 'star':
            # Star topology: All clients connect to representative
            representative = clients[0]
            for client in clients[1:]:
                network['within_domain'][client] = [representative]
                if representative not in network['within_domain']:
                    network['within_domain'][representative] = []
                network['within_domain'][representative].append(client)

    # Step 2: Build between-domain topology
    cluster_ids = list(domain_clusters.keys())
    for i, cluster_id in enumerate(cluster_ids):
        representative = domain_clusters[cluster_id][0]

        next_cluster = cluster_ids[(i + 1) % len(cluster_ids)]
        next_representative = domain_clusters[next_cluster][0]
```

```
        if representative not in network['between_domain']:
            network['between_domain'][representative] = []
        network['between_domain']
[representative].append(next_representative)

    return network
```

Component 2: Hierarchical Gossip Aggregation

2.1 Within-Domain Gossip Aggregation

```python
def within_domain_gossip_aggregation(
    domain_cluster_clients,
    client_lora_matrices,
    client_ranks,
    client_data_quality,
    gossip_round
):
    """
    Perform gossip aggregation within a domain cluster

    Each client exchanges with neighbors and updates its model
    """

    aggregated_models = {}

    for client_id in domain_cluster_clients:
        # Get current model
        current_a, current_b = client_lora_matrices[client_id]

        # Get neighbors (from network topology)
        neighbors = get_neighbors_within_domain(client_id)

        # Collect models from neighbors
        neighbor_models = []
        neighbor_weights = []

        for neighbor_id in neighbors:
            neighbor_a, neighbor_b = client_lora_matrices[neighbor_id]
            neighbor_rank = client_ranks[neighbor_id]
            neighbor_quality = client_data_quality[neighbor_id]

            neighbor_models.append((neighbor_a, neighbor_b))

            # Weight by data quality and rank
            weight = neighbor_quality * np.log(neighbor_rank + 1)
            neighbor_weights.append(weight)

        # Add own model
        own_quality = client_data_quality[client_id]
        own_rank = client_ranks[client_id]
        own_weight = own_quality * np.log(own_rank + 1)
```

```
        neighbor_models.append((current_a, current_b))
        neighbor_weights.append(own_weight)

        # Normalize weights
        neighbor_weights = np.array(neighbor_weights)
        neighbor_weights = neighbor_weights / np.sum(neighbor_weights)

        # Aggregate: Weighted average
        aggregated_a = np.zeros_like(current_a)
        aggregated_b = np.zeros_like(current_b)

        for (neighbor_a, neighbor_b), weight in zip(neighbor_models,
neighbor_weights):
            # Handle heterogeneous ranks using projection
            neighbor_a_proj = project_to_rank(neighbor_a,
current_a.shape[1])
            neighbor_b_proj = project_to_rank(neighbor_b,
current_b.shape[0])

            aggregated_a += weight * neighbor_a_proj
            aggregated_b += weight * neighbor_b_proj

        aggregated_models[client_id] = (aggregated_a, aggregated_b)

    return aggregated_models
```

2.2 Cross-Domain Transfer Learning

```
def cross_domain_transfer_learning(
    domain_clusters,
    domain_lora_modules,
    validation_data_per_domain
):
    """
    Learn cross-domain transfer weights

    Determine how much each domain should influence others
    """

    n_domains = len(domain_clusters)
    transfer_weights = np.zeros((n_domains, n_domains))

    # For each domain as target
    for target_domain_id, target_domain_clients in domain_clusters.items():
        # Get validation data for target domain
        val_data = validation_data_per_domain[target_domain_id]

        # Test how well each domain's LoRA performs on target domain
        performances = []
        source_domain_ids = []

        for source_domain_id, source_lora in domain_lora_modules.items():
            # Evaluate source domain's LoRA on target domain's validation
data
            perf = evaluate_lora_on_domain(source_lora, val_data)
            performances.append(perf)
            source_domain_ids.append(source_domain_id)

        # Softmax to convert to weights
        performances = np.array(performances)
        transfer_weights_for_target = softmax(performances)

        # Store transfer weights
        for source_id, weight in zip(source_domain_ids,
transfer_weights_for_target):
            transfer_weights[source_id, target_domain_id] = weight

    return transfer_weights
```

2.3 Between-Domain Gossip with Transfer Weights

```
def between_domain_gossip_aggregation(
    domain_representatives,
    domain_lora_modules,
    transfer_weights,
    gossip_round
):
    """
    Perform gossip aggregation between domain representatives

    Uses transfer weights to determine how much each domain influences
others
    """

    aggregated_domain_models = {}

    for target_domain_id, target_representative in
domain_representatives.items():
        # Get current model
        current_a, current_b = domain_lora_modules[target_domain_id]

        # Get neighbors (other domain representatives)
        neighbors = get_neighbors_between_domain(target_representative)

        # Collect models from neighbors
        neighbor_models = []
        neighbor_weights = []

        for neighbor_id in neighbors:
            neighbor_domain_id = get_domain_of_client(neighbor_id)
            neighbor_a, neighbor_b = domain_lora_modules[neighbor_domain_id]

            neighbor_models.append((neighbor_a, neighbor_b))

            # Weight by transfer weight
            weight = transfer_weights[neighbor_domain_id, target_domain_id]
            neighbor_weights.append(weight)

        # Add own model
        neighbor_models.append((current_a, current_b))
        neighbor_weights.append(transfer_weights[target_domain_id,
target_domain_id])

        # Normalize weights
```

```
        neighbor_weights = np.array(neighbor_weights)
        neighbor_weights = neighbor_weights / np.sum(neighbor_weights)

        # Aggregate
        aggregated_a = np.zeros_like(current_a)
        aggregated_b = np.zeros_like(current_b)

        for (neighbor_a, neighbor_b), weight in zip(neighbor_models,
neighbor_weights):
            aggregated_a += weight * neighbor_a
            aggregated_b += weight * neighbor_b

        aggregated_domain_models[target_domain_id] = (aggregated_a,
aggregated_b)

    return aggregated_domain_models
```

Component 3: Convergence Analysis

3.1 Convergence Theorem

Theorem: Under standard assumptions (bounded gradients, L-smooth loss), the hierarchical gossip aggregation algorithm converges to a stationary point with rate $O(1/\sqrt{T})$ where T is the number of rounds.

Proof Sketch:

1. Within-domain gossip converges due to gossip matrix properties
2. Between-domain gossip adds bounded error due to transfer weights
3. Combined convergence rate is dominated by slower component

3.2 Convergence Rate Analysis

```
def analyze_convergence_rate(
    loss_history,
    domain_assignments,
    network_topology
):
    """
    Analyze convergence rate empirically
    """

    # Compute loss reduction per round
    loss_reductions = np.diff(loss_history)

    # Fit to theoretical convergence rate O(1/√T)
    rounds = np.arange(1, len(loss_history))
    theoretical_rate = 1.0 / np.sqrt(rounds)

    # Normalize both to [0, 1]
    loss_reductions_norm = (loss_reductions - loss_reductions.min()) /
(loss_reductions.max() - loss_reductions.min())
    theoretical_rate_norm = (theoretical_rate - theoretical_rate.min()) /
(theoretical_rate.max() - theoretical_rate.min())

    # Compute correlation
    correlation = np.corrcoef(loss_reductions_norm, theoretical_rate_norm)
[0, 1]

    return {
        'empirical_rate': loss_reductions,
        'theoretical_rate': theoretical_rate,
        'correlation': correlation
    }
```

Experimental Design

Datasets

Domain 1: Medical Imaging

- Cardiology: 10,000 cardiac images

- Oncology: 8,000 tumor images
- Neurology: 6,000 brain images

Domain 2: Finance

- Retail Banking: 50,000 transaction records
- Investment Banking: 30,000 market records
- Insurance: 20,000 claim records

Domain 3: Scientific Literature

- Physics: 5,000 papers
- Biology: 4,000 papers
- Computer Science: 6,000 papers

Network Topologies

Topology 1: Ring Network

```
C1 ↔ C2 ↔ C3 ↔ ... ↔ C12 ↔ C1
```

Topology 2: Hierarchical Cluster Network

```
Within-domain: Ring
Between-domain: Representative connections
```

Topology 3: Random Regular Graph

```
Each client has exactly k neighbors (e.g., k=4)
```

Baselines

1. **Standard Gossip**: Uniform averaging without domain awareness
2. **Centralized FedAvg**: Central server aggregation (upper bound)

3. **Gossip SGD**: Standard gossip-based SGD
4. **Gossip with Personalization**: Personalized gossip without domain clustering

Metrics

Convergence Metrics:

- Rounds to target accuracy
- Convergence speed
- Convergence stability

Accuracy Metrics:

- In-domain accuracy per client
- Cross-domain generalization accuracy
- Average accuracy across all clients
- Per-domain accuracy variance (fairness)

Communication Metrics:

- Total messages transmitted
- Messages per round
- Communication cost per client

Decentralization Metrics:

- Network diameter (hops between farthest nodes)
- Message complexity per round
- Robustness to client dropout

Implementation Roadmap

Technology Stack

Core Libraries

- PyTorch 2.0+
- Transformers (Hugging Face)
- PEFT
- NumPy
- Scikit-learn

Code Structure

```
project-3-hierarchical-gossip/
├── README.md
├── setup.py
├── requirements.txt
├── src/
│   ├── clustering/
│   │   ├── domain_clustering.py
│   │   └── topology_builder.py
│   ├── aggregation/
│   │   ├── within_domain_gossip.py
│   │   ├── between_domain_gossip.py
│   │   └── transfer_learning.py
│   ├── analysis/
│   │   ├── convergence_analyzer.py
│   │   └── fairness_metrics.py
│   ├── utils/
│   │   ├── metrics.py
│   │   └── visualization.py
│   └── main.py
├── experiments/
└── tests/
```

Timeline and Milestones

Phase	Duration	Milestones
1. Literature Review	Weeks 1-2	Understand gossip algorithms, decentralized FL, domain adaptation
2. Algorithm Design	Weeks 3-4	Design clustering and hierarchical gossip algorithms
3. Theoretical Analysis	Weeks 5-6	Prove convergence guarantees
4. Implementation	Weeks 7-10	Implement all components
5. Experimentation	Weeks 11-16	Run experiments on all topologies and datasets
6. Analysis & Writing	Weeks 17-22	Analyze results, write paper
7. Revision & Submission	Weeks 23-26	Revise based on feedback, submit

---

Expected Outcomes

Research Contributions

1. Hierarchical gossip aggregation algorithm for decentralized federated learning
2. Domain-aware clustering for identifying natural client groups
3. Cross-domain transfer learning for enabling knowledge sharing
4. Convergence analysis with theoretical guarantees
5. Comprehensive evaluation on multiple topologies and datasets

Expected Results

- 20-25% reduction in communication rounds
- 15-20% accuracy improvement over domain-unaware gossip
- Convergence guarantees with theoretical analysis

- Scalable to 100+ clients in decentralized networks

Conclusion

This project presents **Hierarchical Gossip Aggregation for Decentralized Federated Learning with Domain Clustering**. The work enables efficient peer-to-peer communication while explicitly accounting for domain heterogeneity, eliminating the need for a central server.

Key Innovation: Domain-aware clustering combined with hierarchical gossip aggregation that preserves domain-specific knowledge while enabling cross-domain transfer.

Expected Impact: 20-25% reduction in communication rounds, 15-20% accuracy improvement, convergence guarantees, scalability to 100+ clients.

Timeline: 9-11 months from start to publication.